

Reviewer Pack — Minimal Local Indeterminacy in Noncommutative Geometry and C...

Entry ba39a1f65606 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 09:09:38 UTC

Minimal Local Indeterminacy in Noncommutative Geometry and Communication Complexity Rank Inequality

Entry ID: ba39a1f65606 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 09:09:38 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every noncommutative algebra A with a given set of generators, the minimal local indeterminacy ($\text{lind}(A)$) is linearly correlated with the communication complexity rank ($\text{CCrank}(A)$), such that $\text{lind}(A) = \Omega(2^{\text{CCrank}(A)})$.

Rationale (proposer's reasoning):

The structure of the noncommutative algebra A provides a geometric interpretation of the communication process, potentially leading to a lower bound on the rank needed for efficient communication. This correlation could reveal new insights into both fields.

Taxonomy category: Noncommutative_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 599e4624a546c2bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of randomly generated noncommutative algebras with a given set of generators exhibit a correlation coefficient between minimal local indeterminacy ($\text{lind}(A)$) and 2 raised to the power of communication complexity rank ($\text{CCrank}(A)$) greater than or equal to 0.8, without any $\text{CCrank}(A)$ value producing $\text{lind}(A) > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_noncommutative_algebra(n):
        # Simple noncommutative algebra generator for demonstration purposes
        A = [[0] * n for _ in range(n)]
        for i in range(n):
            A[i][i] = 1
        return A

    def compute_lind(A):
        # Minimal local indeterminacy (lind) is a placeholder value
        return random.random()

    def compute_ccrank(A):
        # Communication complexity rank (CCrank) is a placeholder value
        return random.randint(1, 5)

    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []
    instances_tested = 0
    n_max = 0

    for n in n_values:
        A = generate_noncommutative_algebra(n)
        lind_A = compute_lind(A)
        CCrank_A = compute_ccrank(A)

        if CCrank_A == 0:
            continue

        instances_tested += 1
        n_max = max(n_max, n)
        metric_values.append(lind_A / (2 ** CCrank_A))
```

```

if not metric_values:
    return {
        "metric_name": "lind(A) / 2^CCrank(A)",
        "metric_value": 0.0,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

correlation_coefficient = sum((x - mean) * (y - mean) for x, y in zip(metric_values, [2 ** CCrank(A) for A in instances_tested])) / len(metric_values)
mean_metric_value = sum(metric_values) / len(metric_values)

return {
    "metric_name": "lind(A) / 2^CCrank(A)",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8 and max(metric_values) <= 10,
    "counterexample": "" if correlation_coefficient >= 0.8 and max(metric_values) <= 10 else "counterexample"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(x["metric_value"] for x in results) / len(results)
    std_metric_value = math.sqrt(sum((x["metric_value"] - mean_metric_value) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={x['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20e013b6.py", line 81, in <module>
 result = run_trial(seed)

- ```
File "/home/ludo/Scrivania/SEC/research/pvsnr_sandbox/test_20e013b6.py", line 63, in run_trial
 correlation_coefficient = sum((x - mean) * (y - mean) for x, y in zip(metric_values, [2 ** CCrank_A for _ in metric_values])) / (sum((x - mean) ** 2 for x in metric_values) ** 0.5) * math.sqrt(sum((y - mean) ** 2 for y in [2 ** CCrank_A for _ in metric_values]))
NameError: name 'mean' is not defined
```

3. The full LLM transcripts are in `research/audit/ba39a1f65606.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/ba39a1f65606.tar.gz` (if generated)